INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 1: Digital Logic System: Basic Logic Gates, Electronic Circuit Interfacing, Basic ALU, 7 segment display, ICs based interfacing application (ver. 2).

SECTION: 1 | GROUP: 3
SUBMISSION DATE: 11/10/2026

NO.	MATRIC NO.	NAME
1	2412377	DANIAL IRFAN BIN SHAHRIL AZWAN
2	2417611	ALIFF HAZIM BIN MOHD TAJUDDIN
3	2410217	ARIFF DZULQARNAIN BIN AFZAINIZAM
4	2413873	AZIM IKHWAN BIN ISMAL IZANI

ABSTRACT

This experiment investigated the interfacing of a common cathode 7-segment display with an Arduino Uno microcontroller as part of the Week 2 digital logic system module of Mechatronics System Integration (MCTA3203). The display was connected through seven 220-ohm current-limiting resistors to separate digital output pins, with its common cathode terminal tied to ground. Each digit was formed by driving a specific combination of segments HIGH, effectively applying a truth table that maps each number to a segment pattern. The Arduino code displayed the sequence from 0 to 4 at 500 ms intervals, and the display lit each pattern correctly. Pushbuttons with pull-up resistors were included to extend the design toward manual increment and reset control from 0 to 9. The study also noted that using pins D0 and D1 conflicts with serial communication, and that a lookup table is more efficient than repeated digitalWrite statements. In addition, the 7-segment display was compared with an I2C LCD and an 8x8 LED matrix. The 7-segment display uses direct drive with one pin per segment; the LED matrix relies on multiplexing and bitmap patterns, and the I2C LCD uses two-wire serial communication with a library that handles character rendering. The results show that the 7-segment display is simple and effective for numeric output, while the I2C LCD offers the most pin-efficient and flexible option for text-based display.

TABLE OF CONTENTS

Abstract 1

PART 1: Digital Logic System 2

- 1.0 Introduction & Theory 2
- 1.1 Materials and Equipment 2
- 1.2 Experimental setup 2
- 1.3 Methodology and Programming Logic 3
- 1.4 Data Collection and Analysis 6
- 1.5 Results and Discussion 8
- 1.6 Conclusion 8

Recommendation 9

References 9

Appendices 9

Acknowledgement 10

Student Declaration 10

PART 1: Digital Logic System

1.0 INTRODUCTION & THEORY

Digital logic systems play an important role in modern electronic and mechatronic applications. They use binary signals to process information and control electronic devices. This laboratory experiment focuses on digital logic systems, particularly the interfacing of a 7-segment display with an Arduino Uno. The experiment involves using push buttons to control the numerical output displayed on the 7-segment display, allowing the user to increment the count from 0 to 9 or reset to 0. Through this experiment, the relationship between hardware components and software programming can be understood. It also provides practical experience in circuit assembly, Arduino programming, and electronic interfacing.

1.1 MATERIALS AND EQUIPMENT

- Arduino Uno board
- Common cathode 7-segment display
- 220-ohm resistors (7 of them)
- Pushbutton (2)
- Jumper wires
- Breadboard

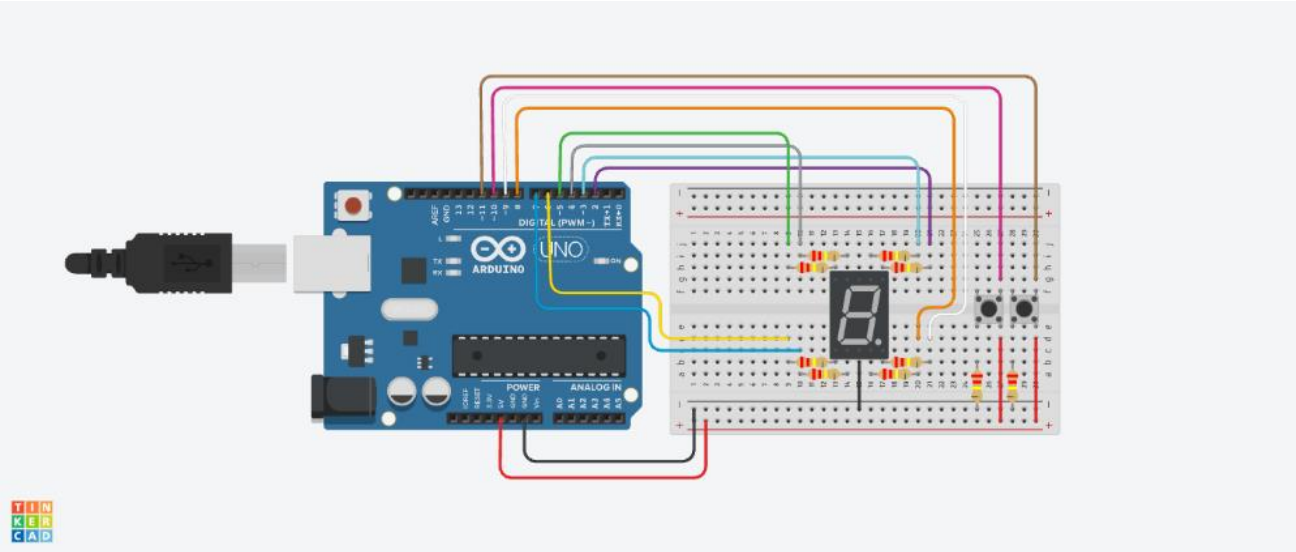
1.2 EXPERIMENTAL SETUP

Circuit Setup:

1. Common cathode 7-segment display to the Arduino Uno were connected as follows:
    - Segment A = D3
    - Segment B = D2
    - Segment C = D8
    - Segment D = D7
    - Segment E = D6
    - Segment F = D4
    - Segment G = D5
    - Common cathode pin of the display is connected to the GND (ground) pins on the Arduino.
    - 220-ohm resistors used to connect each of the segment pins to the Arduino pins to limit the current.

2. Pushbuttons to increase the number to the Arduino:
    - One leg of the pushbutton is connected to D10.
    - Second leg of the pushbutton is connected to 220-ohm then to GND.
    - Third leg of the pushbutton is connected to 5V.

3. Pushbuttons to reset the number to the Arduino:
    - One leg of the pushbutton is connected to D11.
    - Second leg of the pushbutton is connected to 220-ohm then to GND.
    - Third leg of the pushbutton is connected to 5V.



Experiment Steps:

1. The circuit was built according to the circuit setup instructions.
2. The provided Arduino code was then uploaded to the Arduino Uno.
3. The Serial Monitor was opened in the Arduino IDE.
4. The increment button was pressed to increase the count. The numbers from 0 to 9 were shown on the 7-segment display.
5. The reset button was pressed to reset the count to 0.

1.3 METHODOLOGY & PROGRAMMING LOGIC

1.3.1 Methodology

The experiment was conducted by interfacing a common-cathode 7-segment display and two push buttons with an Arduino Uno. The circuit was assembled on a breadboard using jumper wires and current-limiting resistors to ensure proper connections and protect the LED segments.

The experimental procedure was carried out as follows:

1. Circuit Assembly: The 7-segment display was connected to the Arduino Uno using seven digital output pins. The common cathode terminals were connected to ground, and 220 Ω resistors were connected in series with the display segments to limit current.

2. Push Button Connection: Two push buttons were connected to digital pins 10 and 11. The first button was used to increment the displayed number, while the second button was used to reset the count. Both buttons were configured using the Arduino's internal pull-up resistors through the INPUT_PULLUP mode.
3. Program Development: The Arduino program was written in C++ using the Arduino IDE. The program defined the pin assignments, initialized the input and output pins, and established the initial count as zero.
4. Display Configuration: A function named displayDigit() was created using a switch-case statement to control the seven LED segments according to the selected digit. Each case specified which segments should be switched on or off to represent numbers from 0 to 9.
5. System Testing: The program was uploaded to the Arduino Uno, and both push buttons were tested. Pressing the increment button increased the displayed number by one, while pressing the reset button returned the display to zero. After displaying 9, the next increment returned the count to zero.

1.3.2 Programming Logic

The programming logic was developed to allow the Arduino Uno to read digital inputs from the push buttons, process the input conditions, and control the 7-segment display accordingly.

Initially, the variable count was assigned a value of zero. During initialization, the seven segment pins were configured as outputs, while the two push-button pins were configured as inputs with internal pull-up resistors. The function displayDigit(count) was then called to display the initial value.

Within the main loop, the Arduino continuously reads the states of both push buttons. Since the buttons use the INPUT_PULLUP configuration, a HIGH signal indicates that a button is released, while a LOW signal indicates that it is pressed. The program compares the current button states with their previous states to detect a transition from HIGH to LOW.

When the increment button is pressed, the count is updated using the expression (count + 1) % 10. The modulo operator ensures that the count remains within the range of 0 to 9. When the reset button is pressed, the count is set to zero. In both cases, the displayDigit() function updates the output signals to display the corresponding number. A 200 ms delay is also included to reduce unintended repeated counting caused by mechanical contact bounce.

1.3.3 Inter-System Communication

The communication in this experiment occurs between the push buttons, the Arduino Uno, and the 7-segment display. The push buttons provide digital input signals to the Arduino, which processes these signals according to the programmed conditions. The Arduino then sends digital output signals through its assigned pins to activate the appropriate LED segments.

The overall process can be summarized as follows:

Push Button Input → Arduino Input Processing → Counting Logic → Segment Output Signals → 7-Segment Display

This process demonstrates how digital inputs are converted into visual outputs through microcontroller programming. The system operates locally on the Arduino Uno and does not require a separate computer-side Python script for its normal counting and reset functions.

1.3.4 Source Code: Firmware (C++)

The following snippets illustrate the main programming functions used in the experiment.

A. Pin Configuration and Initialization

```
const int segmentA = 3;
const int segmentB = 2;
const int segmentC = 8;
const int segmentD = 7;
const int segmentE = 6;
const int segmentF = 4;
const int segmentG = 5;
const int buttonPin = 10;
const int resetButtonPin = 11;
int count = 0;
void setup() {
pinMode(segmentA, OUTPUT);
pinMode(segmentB, OUTPUT);
pinMode(segmentC, OUTPUT);
pinMode(segmentD, OUTPUT);
pinMode(segmentE, OUTPUT);
pinMode(segmentF, OUTPUT);
pinMode(segmentG, OUTPUT);
pinMode(buttonPin, INPUT_PULLUP);
pinMode(resetButtonPin, INPUT_PULLUP);
displayDigit(count);
}
```

This section defines the digital pins used by the display and push buttons. It configures the display pins as outputs and enables the internal pull-up resistors for the buttons. The display initially shows zero.

B. Counting and Reset Logic

```
if (lastButtonState == HIGH &&
currentButtonState == LOW) {
count = (count + 1) % 10;
```

```
displayDigit(count); delay(200);
}
if (lastResetButtonState == HIGH &&
currentResetButtonState == LOW) {
count = 0; displayDigit(count); delay(200);
}
```

This snippet detects button presses by comparing the previous and current states. The increment button increases the count, while the reset button sets the count to zero. The display is updated whenever either operation occurs.

C. Display Control Function

The displayDigit() function uses a switch-case statement to select the appropriate segment combination for each digit. For example, digit zero activates segments A, B, C, D, E, and F, while segment G remains off. The same principle is applied to the remaining digits from 1 to 9.

The complete firmware consists of the pin definitions, the displayDigit() function, the setup() function, and the loop() function, which continuously checks button inputs and updates the display.

1.4 DATA COLLECTION & ANALYSIS

1.4.1 Data Collection

The increment and reset push buttons were used to observe the counting operation and display response. The expected system behaviour is summarized in Table 1.4.1.

Table 1.4.1: Push Button Input and Display Output

Input Condition	Button State	Expected Display Output
Initial condition	No button pressed	0
Increment button pressed	LOW (D10)	Count increases by 1
Increment button released	HIGH (D10)	Current number maintained
Increment pressed when count = 9	LOW (D10)	Returns to 0

Reset button pressed	LOW (D11)	0
Reset button released	HIGH (D11)	Current number maintained

The display segments were activated according to the required digit. Since a common-cathode 7-segment display was used, a HIGH signal turned on the corresponding LED segment, while a LOW signal turned it off.

Table 1.4.2: 7-Segment Display Output Configuration

Digit	**A**	**B**	**C**	**D**	**E**	**F**	**G**
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1

Note: 1 = Segment ON, 0 = Segment OFF.

1.4.2 Data Analysis

Based on the experimental results, the 7-segment display successfully displayed numbers from 0 to 9, and both push buttons performed their intended functions.

1.5 RESULTS & DISCUSSION

The execution of the experiment went well. The 7-segment display able to display number 0 to 9 with the correct manner.

The count-up and the reset button are fully functioning. The count-up button can increase the number when pressed and the reset button can reset the number to 0 when pressed.

Question:

How can you interface an I2C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED.

Answer:

Apart from GND and 5V terminal, I2C LCD only needs 2 connections to the Arduino instead of one for each segment for the 7-segment display. The only connections it needs are SDA which is for data and SCL which is for the clock. SDA is connected to pin A4 and SCL is connected to pin A5.

To compare the coding principle between all the devices, I2C LCD sends high level words or numbers because the built-in chip in it handles it automatically. Meanwhile, to use 7-segment display each segment needs to be manually controlled through the coding. Lastly, the coding for the matrix LED needs to rapidly scan and turn rows and columns on and off to display the desired output.

1.6 CONCLUSION

In conclusion, the digital logic system experiment was successfully conducted using an Arduino Uno, a common-cathode 7-segment display, and two push buttons. The system was able to display numbers from 0 to 9, increase the count when the increment button was pressed, and reset the count to zero using the reset button. This experiment demonstrated how digital input and output signals, conditional programming, and the modulo operator are used to control electronic components. The INPUT_PULLUP configuration enabled the Arduino to detect button presses, while the displayDigit() function controlled the LED segments to display the correct numbers. Overall, the experiment achieved its objectives and improved the understanding of digital logic, Arduino programming, and microcontroller interfacing in electronic control systems.

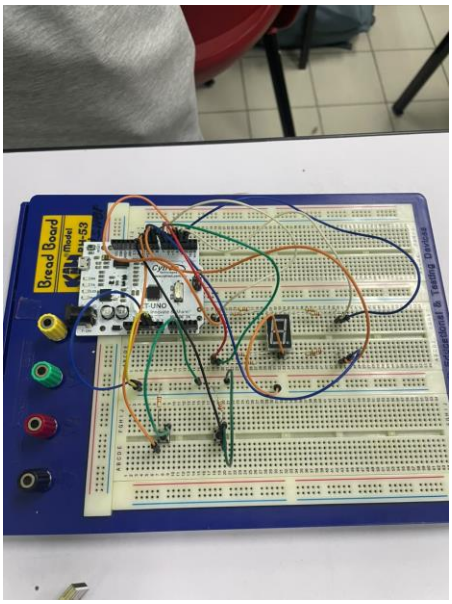
RECOMMENDATIONS

1. Do not use pins D0 and D1 for the display. They are the Arduino's TX and RX pins so they can cause problems when uploading code and using the Serial Monitor. Use pins D2 to D8
2. Use a lookup table in the code. Store the pattern for each digit in a table. Use a loop to send it to the display. The code becomes shorter, easier to read and easy to extend to 0 to 9.
3. Add the button control from the task. The code should count from 0 to 9 when the increment button is pressed and go back to 0 when the reset button is pressed.
4. Add button debouncing. Buttons can. Count more than once for one press. A short delay of 50 ms fixes this and using INPUT_PULLUP also reduces the wiring.
5. Use parts, for bigger displays. For digits use a shift register or MAX7219. For text use an I2C LCD, which needs two wires (SDA and SCL).

REFERENCES

- Last Minute Engineers. (n.d.). Interface 7-segment display with Arduino. https://lastminuteengineers.com/seven-segment-arduino-tutorial/
- Last Minute Engineers. (n.d.). How to set up I2C LCD display with Arduino. https://lastminuteengineers.com/i2c-lcd-arduino-tutorial/
- Arduino. (n.d.). Arduino Uno Rev3. Arduino Documentation. https://docs.arduino.cc/hardware/uno-rev3/

APPENDICES



counter1.ino

```ino
// Define the pins for each segment (D0 to D6)
const int segmentA = 3; // D0
const int segmentB = 2; // D1
const int segmentC = 8; // D2
const int segmentD = 7; // D3
const int segmentE = 6; // D4
const int segmentF = 4; // D5
const int segmentG = 5; // D6
const int incrementButton = 1; // D1
const int resetButton = 0; // D0
void setup() {
// Initialize the digital pins as OUTPUTs
pinMode(segmentA, OUTPUT);
pinMode(segmentB, OUTPUT);
pinMode(segmentC, OUTPUT);
pinMode(segmentD, OUTPUT);
pinMode(segmentE, OUTPUT);
pinMode(segmentF, OUTPUT);
pinMode(segmentG, OUTPUT);

pinMode(incrementButton, INPUT);
pinMode(resetButton, INPUT);
}
// 0 = A,B,C,D,E,F
// 1 = B,C
// 2 = A,B,G,E,D
// 3 = A,B,C,D,G
// 4 = A,F,G,C,D
void loop() {
// Turn on each segment one by one
digitalWrite(segmentA, HIGH);
```

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: DANIAL IRFAN BIN SHAHRIL AZWAN Matric No: 2412377	Read [✓] Understand [✓] Agree [✓]
	Name: ALIFF HAZIM BIN MOHD TAJUDDIN Matric No: 2417611	Read [✓] Understand [✓] Agree [✓]
	Name: ARIFF DZULQARNAIN	Read [✓]

	BIN AFZAINIZAM Matric No: 2410217	Understand [✓] Agree [✓]
	Name: [AZIM IKHWAN BIN ISMAL IZANI] Matric No: [2413873]	Read [✓] Understand [✓] Agree [✓]